COMP 403

Parallel and distributed processing

Lecture 3

5-10-2026

Outline

- Problems in Concurrent Programs
  - Reader and Writers Problem
  - Monitors
    - Monitor for the producer and Consumer Problem

Reader and Writers Problem

Readers and writers are tasks that compete for access to a shared database.

- Readers perform transactions that read data.
- Writers perform transactions that update records.

Synchronization Requirements:

- To maintain data consistency, a writer requires exclusive access to the database (or the specific record) during its transaction.
- When no writers are accessing the database, multiple readers can read records simultaneously.

Readers and Writers Problem - Access Conditions

Mutual Exclusion Rules:

- A reader may access the database only if no writer is currently using it.
- A writer may access the database only if no other task (reader or writer) is currently using it.

Key Point:

The problem reduces to properly synchronizing the access conditions for readers and writers.

Readers and Writers Problem – Solution

- Shared declarations:

```
s: semaphore := (1, ∅);
  -- binary semaphore for mutual exclusion (for protecting no_of_readers variable)
access: semaphore := (1, ∅);
   -- binary semaphore synchronizing access to the database
no_of_readers: natural := 0;
     --  number of readers accessing  simultaneously the database
```

- Readers use semaphore s only to safely increment/decrement no_of_readers and use semaphore access only for the first reader (to block writers) and the last reader (to release writers).
- Writers use access directly to gain exclusive access to the database.

Tasks reader and writer

```
1 task type reader; – – task type specifications
2 task type writer;
3 task body reader is
4 begin
5    loop
6       wait(s);
7       no of readers := no of readers + 1;
8       if no of readers = 1 then wait(access); end if;
9      signal (s);
10      Reading data from the database;
11      wait(s);
12       no of readers := no of readers − 1;
13      if no of readers = 0 then signal (access); end if;
14     signal (s);
15 end loop;  16  end reader;
```

```
17 task body writer is
18 begin
19 loop
20       wait(access);
21          Writing data to the database;
22       signal (access);
23 end loop;
24 end writer;
25 Readers: array(1 .. 100) of reader;
26 Writers: array(1 .. 100) of writer;
```

Reader and Writers Problem: Solution Analysis

Advantage:

- The solution is simple (only 1 semaphore access controls writer exclusion) and easy to implement.

Limitation:

- It is not entirely correct- **writers starvation** may occur.

Cause:

- If a large number of readers continuously issue requests to the database, they dominate access.

Effect:

- Readers repeatedly update the variable no_of_readers (lines 7 and 12) preventing writers from using the database.

Readers and Writers Problem: A Correct Solution

The Declaration of the Solution:

```
s: semaphore := (1, ∅);  -- semaphore for mutual exclusion
reading_allowed: semaphore := (0, ∅);  -- semaphore for readers synchronization
writing_allowed: semaphore := (0, ∅); --semaphore for writers synchronization
nr: natural := 0;        -- number of readers using the database
nw: natural := 0;        -- number of writers using the database
nwr: natural := 0;       -- number of waiting readers
nww: natural := 0;       -- number of waiting writers
```

Reader Task

```
Task type reader;
task body reader is
 begin
    loop
 1.       wait(s); – – pre-protocol of reader
 2.   if nw > 0 then – – writer is accessing the database
 3.          nwr := nwr + 1;
 4.         signal(s);
 5.       wait(reading_allowed);
 6.   end if;
 7.   nr := nr + 1;
 8.  if nwr > 0 then – – there are waiting readers
 9.     nwr := nwr − 1;
 10.    signal(reading_allowed);
 11. else signal(s); end if;
12. Reading data from the database;
 13.    wait(s); nr := nr − 1; – – post-protocol of reader
 14.     if nr = 0 and nww > 0 then;
 15.       nww := nww − 1;
 16.     signal(writing_allowed);
      else signal(s); end if;
end loop;
end reader;
```

Writer Task

```
task type writer
 task body writer is
begin
    loop
       wait(s); – – pre-protocol of writer
      if nr > 0 or nw > 0 then
          nww := nww + 1;
         signal(s);
         wait(writing_allowed);
    end if;
    nw := nw + 1;
    signal(s);
```

Writing data to the database;

```
    wait(s);
     nw := nw − 1; – – post-protocol of writer
    if nwr > 0 then – – there are waiting readers
        nwr := nwr − 1;
       signal(reading_allowed);
    else if nww > 0 then – – there are waiting writer
          nww := nww − 1;
         signal(writing_allowed);
   else
       signal(s); end if;
end loop;
end writer;
```

Correctness Conditions of the Solution

The protocol must ensure the following conditions:

1. Reader Blocking:
   - A new reader is blocked if a writer is waiting for access.
   - Implemented by: (*wait*(*reading_allowed*); line 5)
2. Writer Unblocking:
   - After readers finish using the database, a waiting writer should be unblocked—if one is present.
   - Implemented by:(*signal*(*writing_allowed*); line 16)
3. Reader/Writer Unblocking (After a writer finishes)
   - All waiting readers should be unblocked (if any). ⟶ (signal(reading_allowed))
   - If no readers are waiting, a waiting writer should be unblocked (if any). ⟶(*signal*(*writing_allowed*))

Monitors

A monitor is a high-level synchronization construct that encapsulates:

- Shared data (private — cannot be accessed from outside)
- Procedures that operate on the data
- Synchronization mechanisms (condition variables)

Vital Feature:

- At any given time, only one process can execute a procedure contained in the monitor.

Purpose:

- Enforces mutual exclusion automatically — no need for explicit semaphore operations to protect shared data.

Condition Variables and Synchronization

For synchronization purposes: Condition variables are introduced.

- Two operations act on condition variables
  - wait(a) : blocks the calling process
  - Signal(a) : unblocks a waiting process
- How wait(a) works
  - If a monitor procedure invoked by process P cannot continue because a condition is not satisfied:
  - P executes wait(a) on the corresponding condition variable **a**.
  - P is blocked and inserted into the FIFO queue associated with **a**.
  - P is **no longer active** → does **not** impede access to the monitor.
- This allows another process Q to enter the monitor and perform actions that may satisfy the condition

Signaling and Unblocking

- Process Q may satisfy the condition and execute signal(a).
- Signal(a) unblocks process P waiting in the queue associated with a.

The difficulty:

- Only **one process can be active** inside a monitor at a time.
- The **signaling process is active** → the unblocked process **cannot** become active immediately.
- This can only happen when the **signaling process leaves** the monitor.

Problem:

- Between signal and leaving the monitor, the signaling process may perform further operations.
- One must be sure the signaled condition is **still satisfied** when it leaves.

Recommendation:

- The signal operation should be the **last operation** performed in the monitor procedure.

Condition Variables

- Declared inside the monitor:
  - nonempty: condition; -- signals data is available
  - nonfull: condition; -- signals space is available
- Operations on a condition variable

Operation	Meaning
wait(c)	Suspend the calling process and place it in the queue of condition c; release monitor lock
signal(c)	Wake up one process waiting on condition c (if any); the woken process re-enters the monitor

- Condition variables do not hold a value — they are just queues of waiting processes.

Condition Variables vs. Semaphores

	Condition Variables	Semaphores
Value	Not assigned a value	Have an integer component
Counting	Cannot count operations	Can count wait/signal operations
Nature	Distinct — no stored state	Integer counter + queue
Wait operation	Blocks the process and inserts it into a FIFO queue	Decrements counter; blocks if counter < 0
Signal operation	unblocks a waiting process (if any); otherwise lost	Increments counter; unblocks if processes waiting

Solving the Producer and Consumer Problem with a Monitor

```
1. task Producer; – – task specifications
2 task Consumer;
3 task body Producer is
4     data: data type;
  – – auxiliary variable containing a chunk of data
5 begin
6      loop
7          Produce data;
8          Producer_Consumer.put(data);
9      end loop;
10 end Producer;
```

```
11 task body Consumer is
12 data: data type; – – auxiliary variable
13 begin
14 loop
15          Producer_Consumer.get(data);
16          Consume data;
17 end loop;
18 end Consumer;
```

Producer–Consumer Using a Monitor

Task	Action
Producer	Calls monitor procedure put to insert data
Consumer	Calls monitor procedure get to retrieve data

Mutual exclusion of put/get s guaranteed by the monitor.
Advantage: Producer and consumer need not handle buffer details or full/empty checks — the monitor does it.

The monitor Producer_Consumer

```
1 monitor Producer_Consumer is
2   buffer: array(0 .. n − 1) of data type;
3   in, out : natural := 0; – – buffer indices
4   counter : natural := 0;
   – – counter indicating how many chunks of data are in
   -- the buffer
5  nonempty, nonfull : condition; – – condition variables
6 procedure put (d: in data) is
7 begin
8       if counter = n then wait(nonfull ); end if;
9       buffer(in) := d; – – insert data into the buffer
10      in := (in + 1) mod n;
11     counter := counter + 1;
12    signal (nonempty); – signal data insertion
13 end put ;
```

```
14 procedure get(d: out data) is
15 begin
16      if counter = 0 then wait(nonempty); end if;
17      d := buffer(out ); – – retrieve data from the buffer
18       out := (out + 1) mod n;
19      counter := counter − 1;
20      signal (nonfull ); – – signal that buffer is not full
21 end get;
22 end Producer Consumer;
```

Synchronization Using Condition Variables

- Inserting and retrieving data are protected by the condition variable nonfull and nonempty.

Producer (buffer full):

- If procedure put finds the buffer full:
  - The producer task is **blocked** and placed in the queue associated with nonfull.
  - It is unblocked after the consumer executes get , which performs signal (nonfull).

Consumer (buffer empty):

- If the buffer has no data:
  - The consumer task is blocked and placed in the queue associated with nonempty.
  - It resumes when the producer signals that data have been inserted, signal(nonempty)

Buffer state:

- The degree of filling of the buffer is controlled by the variable counter.